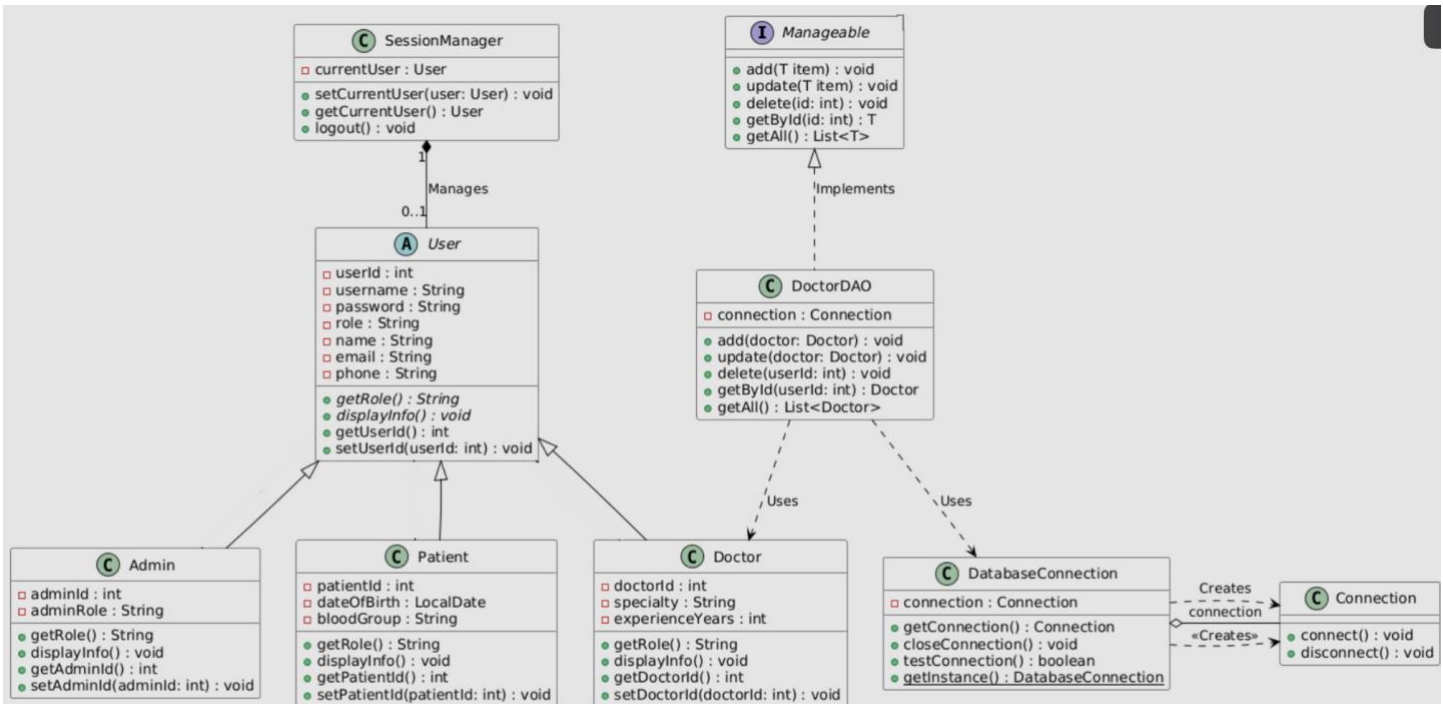


1. Project Description

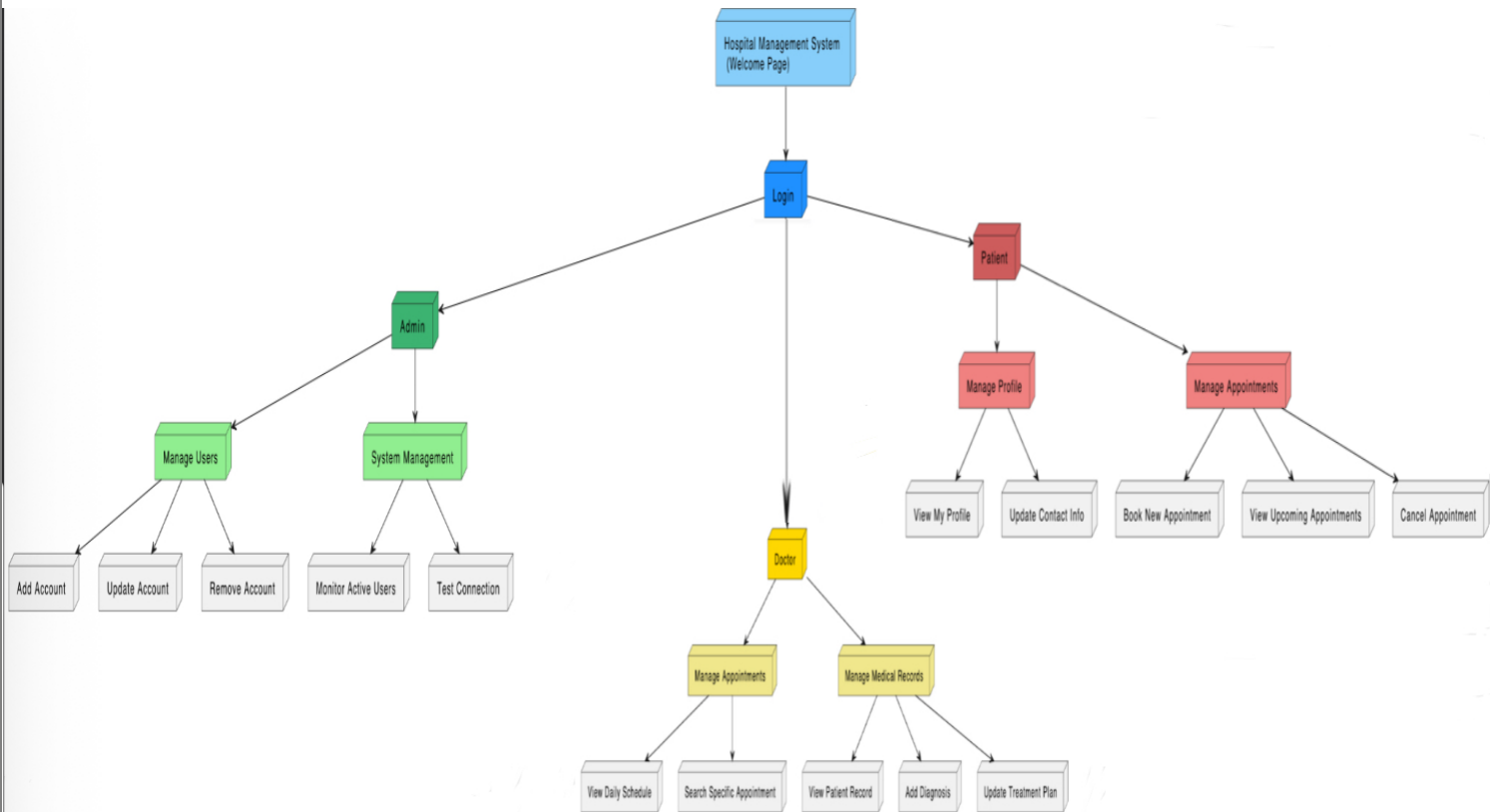
This project presents a Hospital Appointment System designed to simplify the interaction between patients and healthcare services. The system allows patients to create an account, log in securely, and browse a list of available doctors by specialty. Users can schedule appointments by selecting the preferred doctor, date, and time. The system also enables patients to view detailed appointment information and track the status of each booking. Additionally, patients have the ability to cancel appointments when needed, with the interface providing confirmation and status updates. The goal of this project is to create a user-friendly platform that enhances convenience and improves appointment management within hospitals.

2. Class Diagram



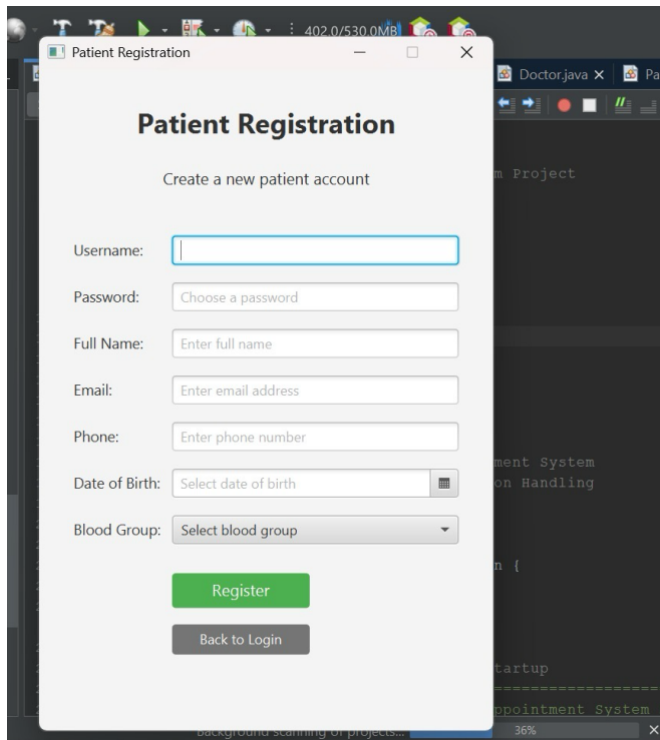
3. GUI Design

System hierarchy



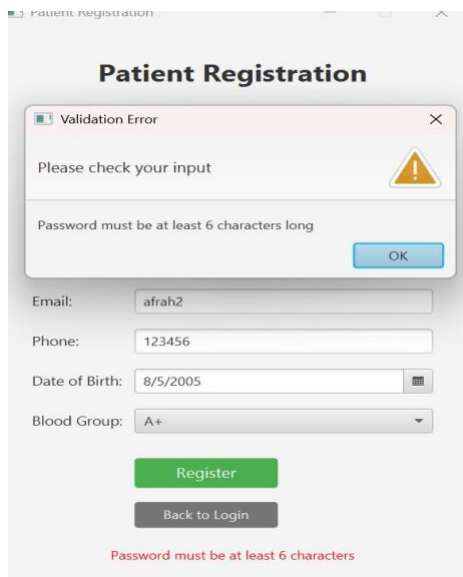
Hospital Appointment System – Patient Module

//Patient Registration Interface



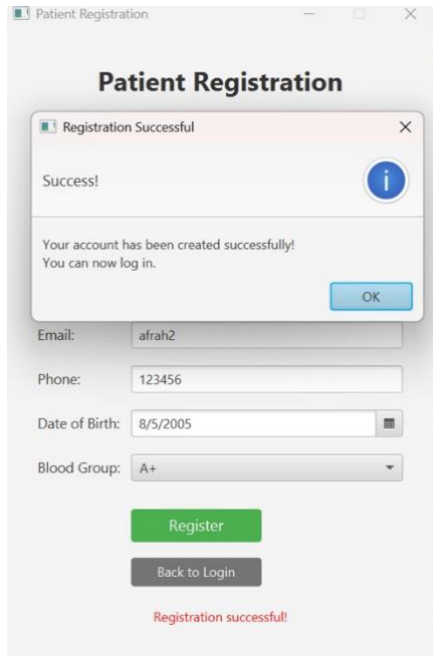
The image shows a 'Patient Registration' window with the title 'Patient Registration' and subtitle 'Create a new patient account'. It contains several input fields: 'Username:' (empty), 'Password:' (placeholder 'Choose a password'), 'Full Name:' (placeholder 'Enter full name'), 'Email:' (placeholder 'Enter email address'), 'Phone:' (placeholder 'Enter phone number'), 'Date of Birth:' (placeholder 'Select date of birth' with a calendar icon), and 'Blood Group:' (placeholder 'Select blood group' with a dropdown arrow). At the bottom are two buttons: a green 'Register' button and a grey 'Back to Login' button. The window is overlaid on a dark background showing code snippets.

//Password Validation Error – Minimum Length Requirement



The image shows the 'Patient Registration' window with a 'Validation Error' dialog box overlaid. The dialog box has a title bar 'Validation Error' and a close button. It contains the text 'Please check your input' and a yellow warning triangle icon. Below this, it says 'Password must be at least 6 characters long' and has an 'OK' button. The background window shows the 'Email:' field with 'afrah2', 'Phone:' with '123456', 'Date of Birth:' with '8/5/2005', and 'Blood Group:' with 'A+'. The 'Register' and 'Back to Login' buttons are still visible. At the bottom of the registration window, there is a red text message: 'Password must be at least 6 characters'.

//Successful Account Creation Message



The image shows a web application window titled "Patient Registration". It contains a form with the following fields: "Email:" (filled with "afrah2"), "Phone:" (filled with "123456"), "Date of Birth:" (filled with "8/5/2005" and a calendar icon), and "Blood Group:" (a dropdown menu showing "A+"). Below the form are two buttons: a green "Register" button and a grey "Back to Login" button. A red message "Registration successful!" is displayed at the bottom of the form. An overlay dialog box titled "Registration Successful" is open, showing a success icon, the text "Success!", and a message: "Your account has been created successfully! You can now log in." with an "OK" button.

Patient Registration

Registration Successful

Success!

Your account has been created successfully!
You can now log in.

OK

Email: afrah2

Phone: 123456

Date of Birth: 8/5/2005

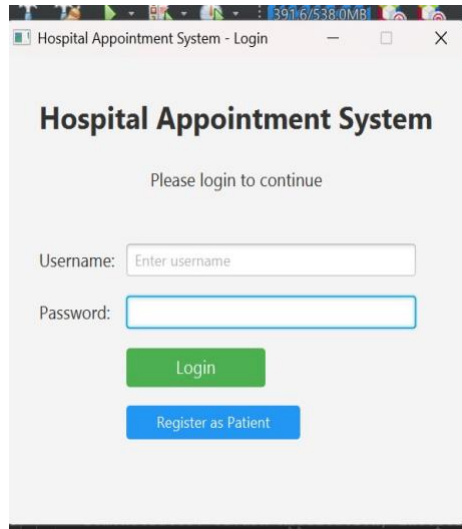
Blood Group: A+

Register

Back to Login

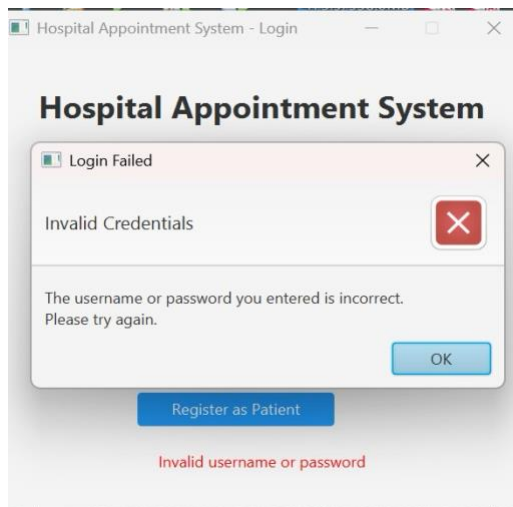
Registration successful!

//User Login Screen for the Hospital Appointment System



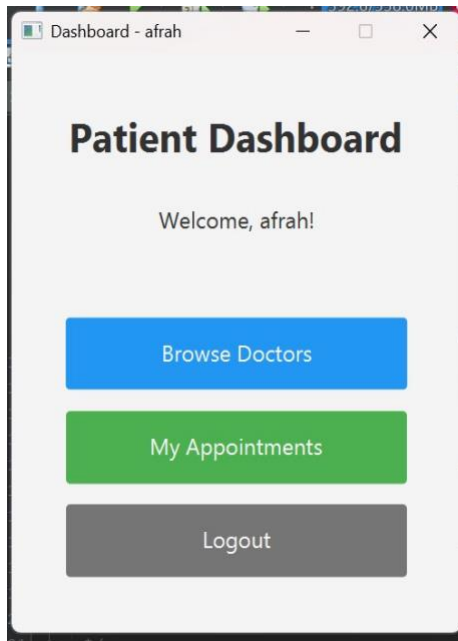
The screenshot shows a web browser window titled "Hospital Appointment System - Login". The page has a light gray background. At the top, the title "Hospital Appointment System" is displayed in bold. Below it, the text "Please login to continue" is centered. There are two input fields: "Username:" with a placeholder "Enter username" and "Password:". Below the password field is a green "Login" button and a blue "Register as Patient" button.

//Login Error Message – Invalid Credentials

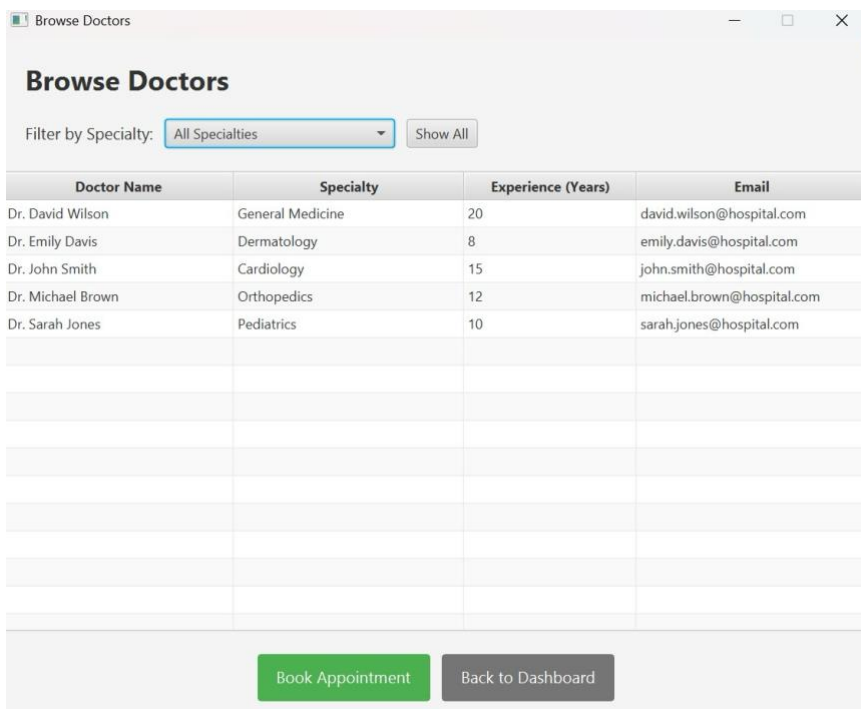


The screenshot shows the same "Hospital Appointment System - Login" web browser window. A modal dialog box titled "Login Failed" is displayed in the center. The dialog has a red header bar and a red "X" icon. The text inside the dialog reads: "Invalid Credentials", "The username or password you entered is incorrect.", and "Please try again." Below the text is a blue "OK" button. In the background, the "Register as Patient" button is visible, and a red error message "Invalid username or password" is displayed at the bottom of the login form.

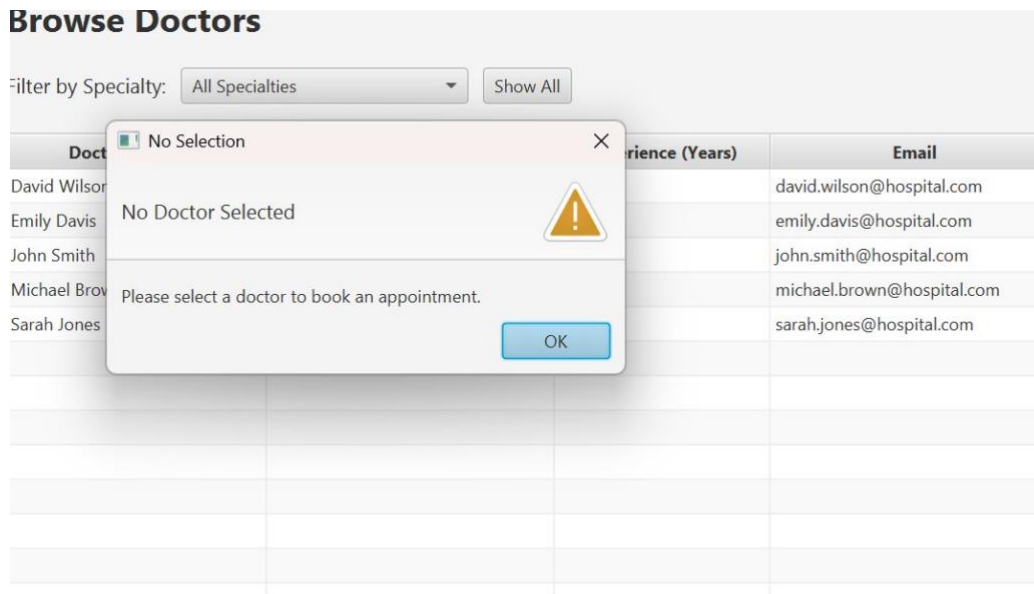
//Patient Dashboard – Main Navigation Screen



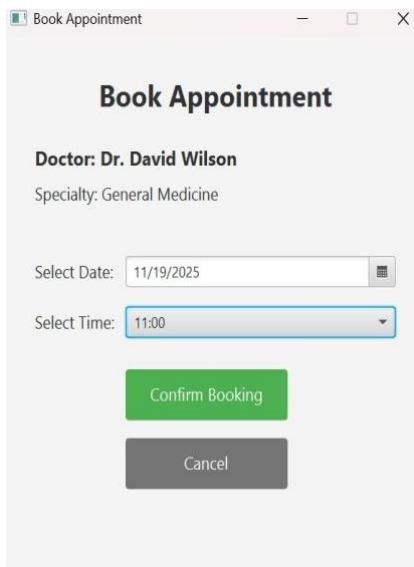
//Doctors Directory – Browse and Filter by Specialty



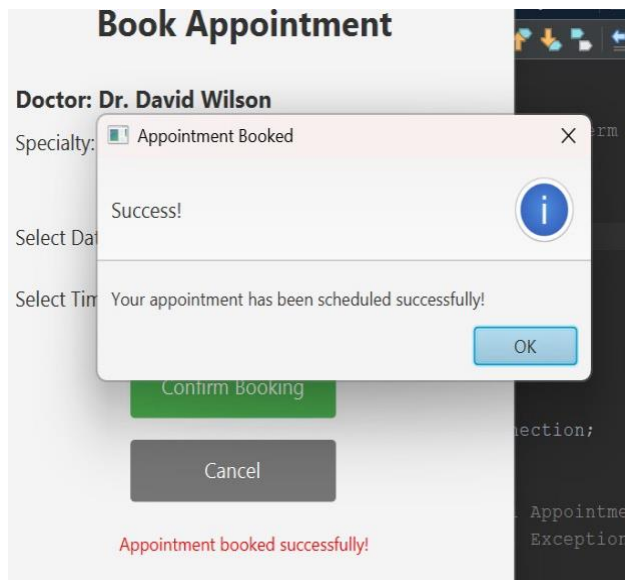
//No Doctor Selected – Warning Message



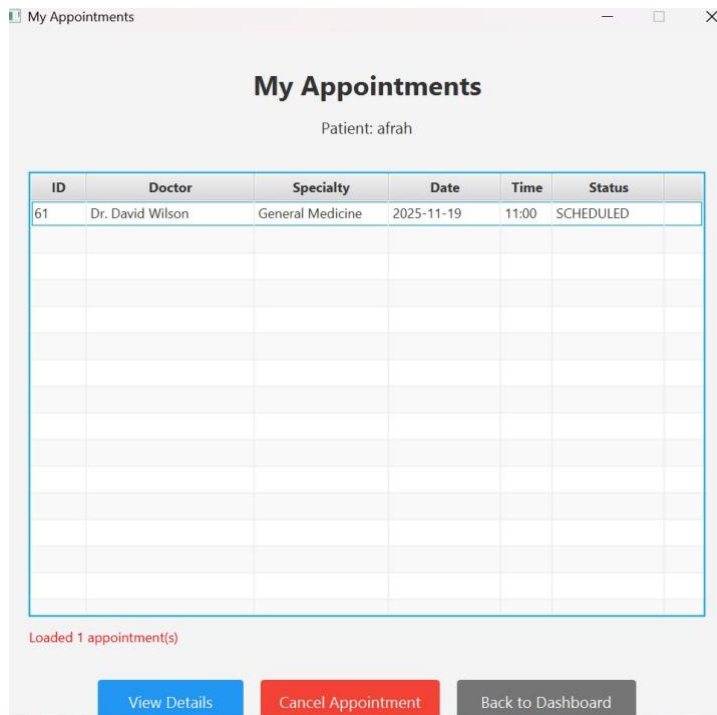
//Select Date and Time for Appointment



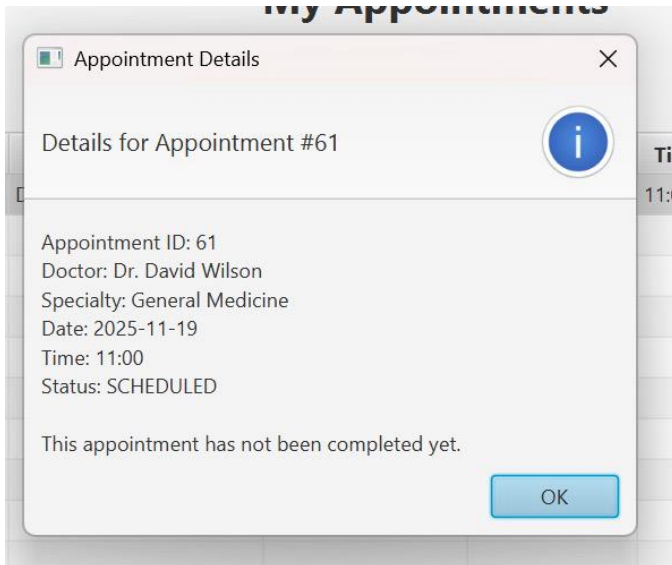
//Appointment Confirmation Message



//My Appointments Page



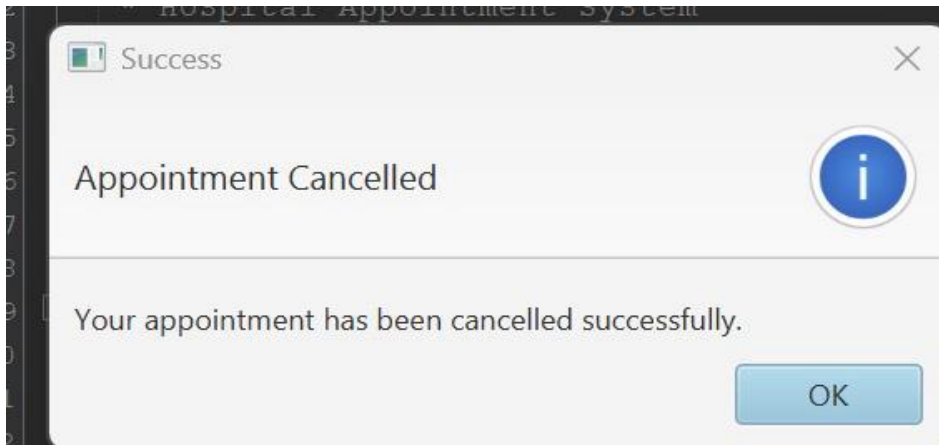
//Appointment Details Window



//Appointment Cancellation Confirmation



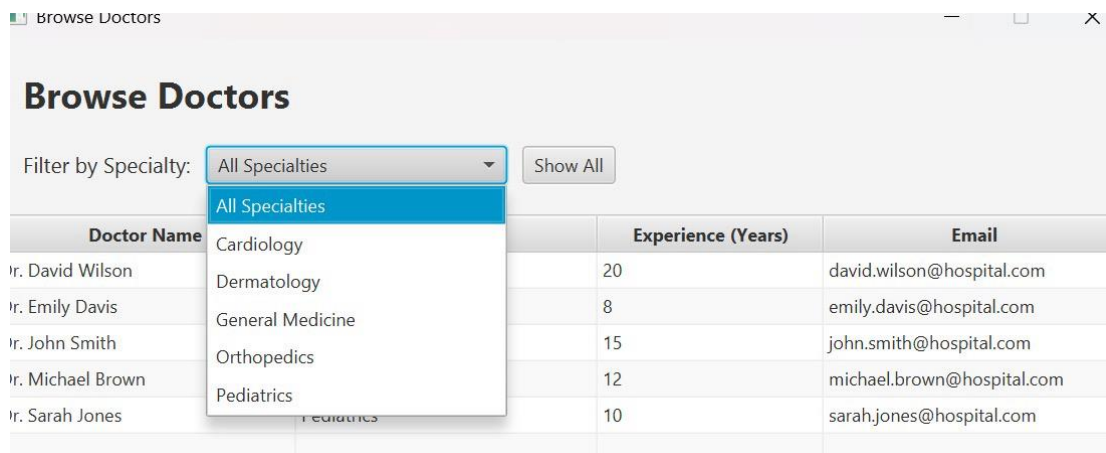
//Appointment Cancellation Success Message



//Updated Appointment Status: CANCELLED

ID	Doctor	Specialty	Date	Time	Status
61	Dr. David Wilson	General Medicine	2025-11-19	11:00	CANCELLED

//Filtering Doctors by Specialty



4.Database Tables

```
-- 1-- USERS Table: Stores all users (Patients, Doctors, Admins)
CREATE TABLE USERS (
    USER_ID NUMBER PRIMARY KEY,
    USERNAME VARCHAR2(50) UNIQUE NOT NULL,
    PASSWORD VARCHAR2(100) NOT NULL,
    ROLE VARCHAR2(20) NOT NULL CHECK (ROLE IN ('PATIENT', 'DOCTOR', 'ADMIN')),
    NAME VARCHAR2(100) NOT NULL,
    EMAIL VARCHAR2(100),
    PHONE VARCHAR2(20) );

-----

-- 2. PATIENTS Table: Stores patient-specific information
CREATE TABLE PATIENTS (
    PATIENT_ID NUMBER PRIMARY KEY,
    USER_ID NUMBER UNIQUE NOT NULL,
    DATE_OF_BIRTH DATE,
    BLOOD_GROUP VARCHAR2(10),
    CONSTRAINT FK_PATIENT_USER FOREIGN KEY (USER_ID)
        REFERENCES USERS(USER_ID) ON DELETE CASCADE );

-----

-- 3. DOCTORS Table: Stores doctor-specific information
CREATE TABLE DOCTORS (
    DOCTOR_ID NUMBER PRIMARY KEY,
    USER_ID NUMBER UNIQUE NOT NULL,
    SPECIALTY VARCHAR2(100) NOT NULL,
    EXPERIENCE_YEARS NUMBER,
    CONSTRAINT FK_DOCTOR_USER FOREIGN KEY (USER_ID)
        REFERENCES USERS(USER_ID) ON DELETE CASCADE );

-----

-- 4. APPOINTMENTS Table: Stores appointment information
CREATE TABLE APPOINTMENTS (
    APPOINTMENT_ID NUMBER PRIMARY KEY,
    PATIENT_ID NUMBER NOT NULL,
    DOCTOR_ID NUMBER NOT NULL,
    APPOINTMENT_DATE DATE NOT NULL,
    APPOINTMENT_TIME VARCHAR2(10) NOT NULL,
    STATUS VARCHAR2(20) DEFAULT 'SCHEDULED' CHECK (STATUS IN ('SCHEDULED', 'COMPLETED',
'CANCELLED')),
    VISIT_NOTES CLOB,
    CONSTRAINT FK_APPT_PATIENT FOREIGN KEY (PATIENT_ID)
        REFERENCES PATIENTS(PATIENT_ID),
    CONSTRAINT FK_APPT_DOCTOR FOREIGN KEY (DOCTOR_ID)
        REFERENCES DOCTORS(DOCTOR_ID));
```

5. Connection and Queries on Database

DatabaseConnection

```
public class DatabaseConnection {
    private static DatabaseConnection instance;
    private Connection connection;
    private static final String DB_URL = "jdbc:oracle:thin:@localhost:1521:XE";
    private static final String DB_USER = "SYSTEM";
    private static final String DB_PASSWORD = "SYSTEM2005";
    private DatabaseConnection() {
        try {
            // Load Oracle JDBC driver
            Class.forName("oracle.jdbc.driver.OracleDriver");
            // Establish connection
```

```

        connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
        System.out.println("Database connection established successfully!");
    } catch (ClassNotFoundException e) {
        System.err.println("Oracle JDBC Driver not found!");
        System.err.println("Please add ojdbc8.jar to the project library.");
        e.printStackTrace();
    } catch (SQLException e) {
        System.err.println("Failed to connect to database!");
        System.err.println("Error: " + e.getMessage());
        e.printStackTrace();
    }
}

public static synchronized DatabaseConnection getInstance() {
    if (instance == null) {
        instance = new DatabaseConnection();
    }
    return instance;
}

public Connection getConnection() throws SQLException {
    // Check if connection is still valid
    if (connection == null || connection.isClosed()) {
        try {
            connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
        } catch (SQLException e) {
            System.err.println("Failed to re-establish database connection!");
            throw e;
        }
    }
    return connection;
}
}

```

DatabaseHandler (Queries on Database)

```

// Login
public static String validateLogin(String username, String password) {
    String sql = "SELECT role FROM USERS WHERE username = ? AND password = ?";
    try {
        Connection conn = DatabaseConnection.getInstance().getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.setString(1, username);
        stmt.setString(2, password);
        ResultSet rs = stmt.executeQuery();

        if (rs.next()) return rs.getString("role");
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return null;
}

// Add Patient
public static boolean addPatient(Patient p) {
    String userSql = "INSERT INTO USERS (USER_ID, USERNAME, PASSWORD, ROLE, NAME, EMAIL, PHONE) "
        + "VALUES (USER_SEQ.NEXTVAL, ?, ?, 'PATIENT', ?, ?, ?)";
    String patientSql = "INSERT INTO PATIENTS (PATIENT_ID, USER_ID, DATE_OF_BIRTH, BLOOD_GROUP) "
        + "VALUES (PATIENT_SEQ.NEXTVAL, USER_SEQ.CURRVAL, ?, ?)";

    try {
        Connection conn = DatabaseConnection.getInstance().getConnection();

        PreparedStatement u = conn.prepareStatement(userSql);
        u.setString(1, p.getUsername());
        u.setString(2, p.getPassword());
        u.setString(3, p.getName());
        u.setString(4, p.getEmail());
        u.setString(5, p.getPhone());
        u.executeUpdate();

        PreparedStatement pt = conn.prepareStatement(patientSql);
    }
}

```

```

        pt.setDate(1, p.getDateOfBirth() != null ? Date.valueOf(p.getDateOfBirth()) : null);
        pt.setString(2, p.getBloodGroup());
        pt.executeUpdate();

        return true;
    } catch (SQLException e) { return false; }
}

// Add Doctor
public static boolean addDoctor(Doctor d) {
    String userSql = "INSERT INTO USERS (USER_ID, USERNAME, PASSWORD, ROLE, NAME, EMAIL, PHONE) "
        + "VALUES (USER_SEQ.NEXTVAL, ?, ?, 'DOCTOR', ?, ?, ?)";
    String doctorSql = "INSERT INTO DOCTORS (DOCTOR_ID, USER_ID, SPECIALTY, EXPERIENCE_YEARS) "
        + "VALUES (DOCTOR_SEQ.NEXTVAL, USER_SEQ.CURRVAL, ?, ?)";

    try {
        Connection conn = DatabaseConnection.getInstance().getConnection();

        PreparedStatement u = conn.prepareStatement(userSql);
        u.setString(1, d.getUsername());
        u.setString(2, d.getPassword());
        u.setString(3, d.getName());
        u.setString(4, d.getEmail());
        u.setString(5, d.getPhone());
        u.executeUpdate();

        PreparedStatement dt = conn.prepareStatement(doctorSql);
        dt.setString(1, d.getSpecialty());
        dt.setInt(2, d.getExperienceYears());
        dt.executeUpdate();

        return true;
    } catch (SQLException e) { return false; }
}

// Add Appointment
public static boolean addAppointment(Appointment a) {
    String sql =
        "INSERT INTO APPOINTMENTS (APPOINTMENT_ID, PATIENT_ID, DOCTOR_ID, APPOINTMENT_DATE,
APPOINTMENT_TIME, STATUS) "
        + "VALUES (APPOINTMENT_SEQ.NEXTVAL, ?, ?, ?, ?, ?)";

    try {
        Connection conn = DatabaseConnection.getInstance().getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);

        stmt.setInt(1, a.getPatientId());
        stmt.setInt(2, a.getDoctorId());
        stmt.setDate(3, Date.valueOf(a.getAppointmentDate()));
        stmt.setString(4, a.getAppointmentTime());
        stmt.setString(5, a.getStatus());
        stmt.executeUpdate();

        return true;
    } catch (SQLException e) { return false; }
}

// Get Appointment by ID
public static Appointment getAppointment(int id) {
    String sql = "SELECT * FROM APPOINTMENTS WHERE APPOINTMENT_ID = ?";
    try {
        Connection conn = DatabaseConnection.getInstance().getConnection();
        PreparedStatement stmt = conn.
        prepareStatement(sql);
        stmt.setInt(1, id);
        ResultSet rs = stmt.executeQuery();

        if (rs.next()) {

```

```

        return new Appointment(
            rs.getInt("APPOINTMENT_ID"),
            rs.getInt("PATIENT_ID"),
            rs.getInt("DOCTOR_ID"),
            rs.getDate("APPOINTMENT_DATE").toLocalDate(),
            rs.getString("APPOINTMENT_TIME"),
            rs.getString("STATUS"),
            rs.getString("VISIT_NOTES")
        );
    }
} catch (SQLException e) { e.printStackTrace(); }
return null;
}

// Cancel Appointment
public static boolean cancelAppointment(int id) {
    String sql = "UPDATE APPOINTMENTS SET STATUS='CANCELLED' WHERE APPOINTMENT_ID=?";
    try {
        Connection conn = DatabaseConnection.getInstance().getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.setInt(1, id);
        stmt.executeUpdate();
        return true;
    } catch (SQLException e) { return false; }
}

// Delete Appointment
public static boolean deleteAppointment(int id) {
    String sql = "DELETE FROM APPOINTMENTS WHERE APPOINTMENT_ID=?";
    try {
        Connection conn = DatabaseConnection.getInstance().getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.setInt(1, id);
        stmt.executeUpdate();
        return true;
    } catch (SQLException e) { return false; }
}
}

```

6. Appendix

Classes

```
public abstract class User {
    // Private attributes - Encapsulation
    private int userId;
    private String username;
    private String password;
    private String role;
    private String name;
    private String email;
    private String phone;
    public User() {
    }
    public User(int userId, String username, String password, String role,
        String name, String email, String phone) {
        this.userId = userId;
        this.username = username;
        this.password = password;
        this.role = role;
        this.name = name;
        this.email = email;
        this.phone = phone;
    }
    public User(String username, String password, String role,
        String name, String email, String phone) {
        this.username = username;
        this.password = password;
        this.role = role;
        this.name = name;
        this.email = email;
        this.phone = phone;
    }
    public abstract String getRole();

    public abstract void displayInfo();

    public int getUserId() {
        return userId;
    }
    public void setUserId(int userId) {
        this.userId = userId;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }
}
```

```

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPhone() {
        return phone;
    }

    public void setPhone(String phone) {
        this.phone = phone;
    }

    @Override
    public String toString() {
        return "User{" +
            "userId=" + userId +
            ", username=" + username + "\" +
            ", role=" + role + "\" +
            ", name=" + name + "\" +
            ", email=" + email + "\" +
            ", phone=" + phone + "\" +
            '}'";
    }
}
-----
public class Admin extends User {
    private int adminId;
    public Admin() {
        super();
    }
    public Admin(int userId, String username, String password, String name,
        String email, String phone, int adminId) {
        super(userId, username, password, "ADMIN", name, email, phone);
        this.adminId = adminId;
    }
    public Admin(String username, String password, String name,
        String email, String phone) {
        super(username, password, "ADMIN", name, email, phone);
    }
    @Override
    public String getRole() {
        return "ADMIN";
    }
    @Override
    public void displayInfo() {
        System.out.println("=== Administrator Information ===");
        System.out.println("Admin ID: " + adminId);
        System.out.println("Name: " + getName());
        System.out.println("Email: " + getEmail());
        System.out.println("Phone: " + getPhone());
    }
    public int getAdminId() {
        return adminId;
    }
    public void setAdminId(int adminId) {
        this.adminId = adminId;
    }
    @Override
    public String toString() {
        return "Admin{" +
            "adminId=" + adminId +
            ", userId=" + getUserId() +
            ", name=" + getName() + "\" +
            '}'";
    }
}
-----
public class Doctor extends User {

```



```

private int doctorId;
private String specialty;
private int experienceYears;
public Doctor() {
    super();
}
public Doctor(int userId, String username, String password, String name,
    String email, String phone, int doctorId,
    String specialty, int experienceYears) {
    super(userId, username, password, "DOCTOR", name, email, phone);
    this.doctorId = doctorId;
    this.specialty = specialty;
    this.experienceYears = experienceYears;
}
public Doctor(String username, String password, String name,
    String email, String phone, String specialty,
    int experienceYears) {
    super(username, password, "DOCTOR", name, email, phone);
    this.specialty = specialty;
    this.experienceYears = experienceYears;
}
@Override
public String getRole() {
    return "DOCTOR";
}
@Override
public void displayInfo() {
    System.out.println("=== Doctor Information ===");
    System.out.println("Doctor ID: " + doctorId);
    System.out.println("Name: " + getName());
    System.out.println("Email: " + getEmail());
    System.out.println("Phone: " + getPhone());
    System.out.println("Specialty: " + specialty);
    System.out.println("Experience: " + experienceYears + " years");
}
public int getDoctorId() {
    return doctorId;
}
public void setDoctorId(int doctorId) {
    this.doctorId = doctorId;
}
public String getSpecialty() {
    return specialty;
}
public void setSpecialty(String specialty) {
    this.specialty = specialty;
}
public int getExperienceYears() {
    return experienceYears;
}
public void setExperienceYears(int experienceYears) {
    this.experienceYears = experienceYears;
}
@Override
public String toString() {
    return "Doctor{" +
        "doctorId=" + doctorId +
        ", userId=" + getUserId() +
        ", name=" + getName() + " +
        ", specialty=" + specialty + " +
        ", experienceYears=" + experienceYears +
        '}';
}
}
}

```

```

-----
public class Patient extends User {

```

```

    private int patientId;
    private LocalDate dateOfBirth;
    private String bloodGroup;
    public Patient() {

```

```

    super();}
public Patient(int userId, String username, String password, String name,
               String email, String phone, int patientId,
               LocalDate dateOfBirth, String bloodGroup) {
    super(userId, username, password, "PATIENT", name, email, phone);
    this.patientId = patientId;
    this.dateOfBirth = dateOfBirth;
    this.bloodGroup = bloodGroup;
}
public Patient(String username, String password, String name,
               String email, String phone, LocalDate dateOfBirth,
               String bloodGroup) {
    super(username, password, "PATIENT", name, email, phone);
    this.dateOfBirth = dateOfBirth;
    this.bloodGroup = bloodGroup;
}
@Override
public String getRole() {
    return "PATIENT";
}
@Override
public void displayInfo() {
    System.out.println("=== Patient Information ===");
    System.out.println("Patient ID: " + patientId);
    System.out.println("Name: " + getName());
    System.out.println("Email: " + getEmail());
    System.out.println("Phone: " + getPhone());
    System.out.println("Date of Birth: " + dateOfBirth);
    System.out.println("Blood Group: " + bloodGroup);
}
public int getPatientId() {
    return patientId;
}
public void setPatientId(int patientId) {
    this.patientId = patientId;
}
public LocalDate getDateOfBirth() {
    return dateOfBirth;
}
public void setDateOfBirth(LocalDate dateOfBirth) {
    this.dateOfBirth = dateOfBirth;
}
public String getBloodGroup() {
    return bloodGroup;
}
public void setBloodGroup(String bloodGroup) {
    this.bloodGroup = bloodGroup;
}
@Override
public String toString() {
    return "Patient{" +
        "patientId=" + patientId +
        ", userId=" + getUserId() +
        ", name=" + getName() + "\n" +
        ", dateOfBirth=" + dateOfBirth +
        ", bloodGroup=" + bloodGroup + "\n" +
        '}';
}
}

```

```

-----
public class Appointment {
    private int appointmentId;
    private int patientId;
    private int doctorId;
    private LocalDate appointmentDate;
    private String appointmentTime;
    private String status; // SCHEDULED, COMPLETED, CANCELLED
    private String visitNotes;
    private Patient patient;
    private Doctor doctor;
    private String doctorName;
    private String specialty;
    private String patientName;
    private String patientPhone;
    public Appointment() {}
    public Appointment(int appointmentId, int patientId, int doctorId,
                       LocalDate appointmentDate, String appointmentTime,
                       String status, String visitNotes) {
        this.appointmentId = appointmentId;
        this.patientId = patientId;
    }
}

```

```

        this.doctorId = doctorId;
        this.appointmentDate = appointmentDate;
        this.appointmentTime = appointmentTime;
        this.status = status;
        this.visitNotes = visitNotes; }
public Appointment(int patientId, int doctorId, LocalDate appointmentDate,
        String appointmentTime, String status) {
    this.patientId = patientId;
    this.doctorId = doctorId;
    this.appointmentDate = appointmentDate;
    this.appointmentTime = appointmentTime;
    this.status = status; }
public Appointment(Patient patient, Doctor doctor, LocalDate appointmentDate,
        String appointmentTime) {
    this.patient = patient;
    this.doctor = doctor;
    this.patientId = patient.getId();
    this.doctorId = doctor.getId();
    this.appointmentDate = appointmentDate;
    this.appointmentTime = appointmentTime;
    this.status = "SCHEDULED";}
public int getId() {
    return appointmentId; }
public void setId(int appointmentId) {
    this.appointmentId = appointmentId;
}

public int getPatientId() {
    return patientId;
}

public void setPatientId(int patientId) {
    this.patientId = patientId;
}

public int getDoctorId() {
    return doctorId;
}

public void setDoctorId(int doctorId) {
    this.doctorId = doctorId;
}

public LocalDate getAppointmentDate() {
    return appointmentDate;
}

public void setAppointmentDate(LocalDate appointmentDate) {
    this.appointmentDate = appointmentDate;
}
public String getAppointmentTime() {
    return appointmentTime;
}
public void setAppointmentTime(String appointmentTime) {
    this.appointmentTime = appointmentTime;
}
public String getStatus() {
    return status;
}
public void setStatus(String status) {
    this.status = status;
}
public String getVisitNotes() {
    return visitNotes;
}
public void setVisitNotes(String visitNotes) {

```

```

        this.visitNotes = visitNotes;
    }
    public Patient getPatient() {
        return patient;
    }
    public void setPatient(Patient patient) {
        this.patient = patient;
        this.patientId = patient.getPatientId();
    }
    public Doctor getDoctor() {
        return doctor;
    }
    public void setDoctor(Doctor doctor) {
        this.doctor = doctor;
        this.doctorId = doctor.getDoctorId();
    }
    public String getDoctorName() {
        return doctorName;
    }
    public void setDoctorName(String doctorName) {
        this.doctorName = doctorName;
    }
    public String getSpecialty() {
        return specialty;
    }
    public void setSpecialty(String specialty) {
        this.specialty = specialty;
    }
    public String getPatientName() {
        return patientName;
    }
    public void setPatientName(String patientName) {
        this.patientName = patientName;
    }
    public String getPatientPhone() {
        return patientPhone;
    }
    public void setPatientPhone(String patientPhone) {
        this.patientPhone = patientPhone;
    }
    public void markCompleted(String notes) {
        this.status = "COMPLETED";
        this.visitNotes = notes;
    }
    public void cancel() {
        this.status = "CANCELLED";
    }
    public boolean isScheduled() {
        return "SCHEDULED".equals(this.status);
    }
    public boolean isCompleted() {
        return "COMPLETED".equals(this.status);
    }
    public boolean isCancelled() {
        return "CANCELLED".equals(this.status);
    }
    public String toString() {
        return "Appointment{" +
            "appointmentId=" + appointmentId +
            ", patientId=" + patientId +
            ", doctorId=" + doctorId +
            ", appointmentDate=" + appointmentDate +
            ", appointmentTime=" + appointmentTime + "\" +
            ", status=" + status + "\" +
            '}'
    }
}-----

```

Interfaces Controllers

```

public interface Manageable<T> {
    void add(T item) throws SQLException;
    void update(T item) throws SQLException;
    void delete(int id) throws SQLException;
    T getById(int id) throws SQLException;
    ArrayList<T> getAll() throws SQLException;
}

```

```

-----
public class AppointmentDAO implements Manageable<Appointment> {
    private Connection connection;
    private PatientDAO patientDAO;
}

```

```

private DoctorDAO doctorDAO;
public AppointmentDAO() {
    try {
        this.connection = DatabaseConnection.getInstance().getConnection();
        this.patientDAO = new PatientDAO();
        this.doctorDAO = new DoctorDAO();
    } catch (SQLException e) {
        System.err.println("Failed to get database connection in AppointmentDAO!");
        e.printStackTrace();
    }
}

@Override
public void add(Appointment appointment) throws SQLException {
    String sql = "INSERT INTO APPOINTMENTS (APPOINTMENT_ID, PATIENT_ID, DOCTOR_ID, " +
        "APPOINTMENT_DATE, APPOINTMENT_TIME, STATUS) " +
        "VALUES (APPOINTMENT_SEQ.NEXTVAL, ?, ?, ?, ?, ?)";
    try {
        PreparedStatement stmt = connection.prepareStatement(sql);
        stmt.setInt(1, appointment.getPatientId());
        stmt.setInt(2, appointment.getDoctorId());
        stmt.setDate(3, Date.valueOf(appointment.getAppointmentDate()));
        stmt.setString(4, appointment.getAppointmentTime());
        stmt.setString(5, appointment.getStatus());
        stmt.executeUpdate();
        stmt.close();
        System.out.println("Appointment added successfully!");
    } catch (SQLException e) {
        System.err.println("Error adding appointment: " + e.getMessage());
        throw e;
    }
}

@Override
public void update(Appointment appointment) throws SQLException {
    String sql = "UPDATE APPOINTMENTS SET PATIENT_ID=?, DOCTOR_ID=?, " +
        "APPOINTMENT_DATE=?, APPOINTMENT_TIME=?, STATUS=?, VISIT_NOTES=? " +
        "WHERE APPOINTMENT_ID=?";
    try {
        PreparedStatement stmt = connection.prepareStatement(sql);
        stmt.setInt(1, appointment.getPatientId());
        stmt.setInt(2, appointment.getDoctorId());
        stmt.setDate(3, Date.valueOf(appointment.getAppointmentDate()));
        stmt.setString(4, appointment.getAppointmentTime());
        stmt.setString(5, appointment.getStatus());
        stmt.setString(6, appointment.getVisitNotes());
        stmt.setInt(7, appointment.getAppointmentId());
        stmt.executeUpdate();
        stmt.close();

        System.out.println("Appointment updated successfully!");
    } catch (SQLException e) {
        System.err.println("Error updating appointment: " + e.getMessage());
        throw e;
    }
}

@Override
public void delete(int appointmentId) throws SQLException {
    String sql = "DELETE FROM APPOINTMENTS WHERE APPOINTMENT_ID=?";
    try {
        PreparedStatement stmt = connection.prepareStatement(sql);
        stmt.setInt(1, appointmentId);
        int rowsAffected = stmt.executeUpdate();
        stmt.close();
    }
}

```

```

        if (rowsAffected > 0) {
            System.out.println("Appointment deleted successfully!");
        } else {
            System.out.println("Appointment not found!");
        }
    }

    } catch (SQLException e) {
        System.err.println("Error deleting appointment: " + e.getMessage());
        throw e;
    }
}

@Override
public Appointment getById(int appointmentId) throws SQLException {
    String sql = "SELECT * FROM APPOINTMENTS WHERE APPOINTMENT_ID = ?";

    try {
        PreparedStatement stmt = connection.prepareStatement(sql);
        stmt.setInt(1, appointmentId);
        ResultSet rs = stmt.executeQuery();

        Appointment appointment = null;
        if (rs.next()) {
            appointment = extractAppointmentFromResultSet(rs);
        }

        rs.close();
        stmt.close();
        return appointment;

    } catch (SQLException e) {
        System.err.println("Error getting appointment by ID: " + e.getMessage());
        throw e;
    }
}

@Override
public ArrayList<Appointment> getAll() throws SQLException {
    String sql = "SELECT * FROM APPOINTMENTS ORDER BY APPOINTMENT_DATE, APPOINTMENT_TIME";
    ArrayList<Appointment> appointments = new ArrayList<>();

    try {
        Statement stmt = connection.createStatement();
        ResultSet rs = stmt.executeQuery(sql);

        while (rs.next()) {
            Appointment appointment = extractAppointmentFromResultSet(rs);
            appointments.add(appointment);
        }
        rs.close();
        stmt.close();
        return appointments;
    } catch (SQLException e) {
        System.err.println("Error getting all appointments: " + e.getMessage());
        throw e;
    }
}

public ArrayList<Appointment> getByPatientId(int patientId) throws SQLException {
    String sql = "SELECT * FROM APPOINTMENTS WHERE PATIENT_ID = ? " +
        "ORDER BY APPOINTMENT_DATE DESC, APPOINTMENT_TIME DESC";
    ArrayList<Appointment> appointments = new ArrayList<>();

    try {
        PreparedStatement stmt = connection.prepareStatement(sql);
        stmt.setInt(1, patientId);
        ResultSet rs = stmt.executeQuery();
        while (rs.next()) {
            Appointment appointment = extractAppointmentFromResultSet(rs);
            appointments.add(appointment);
        }
        rs.close();
        stmt.close();
    }
}

```

```

        return appointments;
    } catch (SQLException e) {
        System.err.println("Error getting appointments by patient ID: " + e.getMessage());
        throw e;
    }
}

public ArrayList<Appointment> getByDoctorId(int doctorId) throws SQLException {
    String sql = "SELECT * FROM APPOINTMENTS WHERE DOCTOR_ID = ? " +
        "ORDER BY APPOINTMENT_DATE, APPOINTMENT_TIME";
    ArrayList<Appointment> appointments = new ArrayList<>();

    try {
        PreparedStatement stmt = connection.prepareStatement(sql);
        stmt.setInt(1, doctorId);
        ResultSet rs = stmt.executeQuery();

        while (rs.next()) {
            Appointment appointment = extractAppointmentFromResultSet(rs);
            appointments.add(appointment);
        }

        rs.close();
        stmt.close();
        return appointments;
    } catch (SQLException e) {
        System.err.println("Error getting appointments by doctor ID: " + e.getMessage());
        throw e;
    }
}

public ArrayList<Appointment> getByDoctorAndDate(int doctorId, LocalDate date) throws SQLException {
    String sql = "SELECT * FROM APPOINTMENTS WHERE DOCTOR_ID = ? AND APPOINTMENT_DATE = ? " +
        "ORDER BY APPOINTMENT_TIME";
    ArrayList<Appointment> appointments = new ArrayList<>();

    try {
        PreparedStatement stmt = connection.prepareStatement(sql);
        stmt.setInt(1, doctorId);
        stmt.setDate(2, Date.valueOf(date));
        ResultSet rs = stmt.executeQuery();
        while (rs.next()) {
            Appointment appointment = extractAppointmentFromResultSet(rs);
            appointments.add(appointment);
        }
        rs.close();
        stmt.close();
        return appointments;
    } catch (SQLException e) {
        System.err.println("Error getting appointments by doctor and date: " + e.getMessage());
        throw e;
    }
}

public ArrayList<Appointment> getByStatus(String status) throws SQLException {
    String sql = "SELECT * FROM APPOINTMENTS WHERE STATUS = ? " +
        "ORDER BY APPOINTMENT_DATE, APPOINTMENT_TIME";
    ArrayList<Appointment> appointments = new ArrayList<>();

    try {
        PreparedStatement stmt = connection.prepareStatement(sql);
        stmt.setString(1, status);
        ResultSet rs = stmt.executeQuery();
        while (rs.next()) {
            Appointment appointment = extractAppointmentFromResultSet(rs);
            appointments.add(appointment);
        }
        rs.close();
        stmt.close();
        return appointments;
    } catch (SQLException e) {
        System.err.println("Error getting appointments by status: " + e.getMessage());
        throw e;
    }
}

```

```

    }
}public void cancelAppointment(int appointmentId) throws SQLException {
    String sql = "UPDATE APPOINTMENTS SET STATUS='CANCELLED' WHERE APPOINTMENT_ID=?";
    try {
        PreparedStatement stmt = connection.prepareStatement(sql);
        stmt.setInt(1, appointmentId);
        stmt.executeUpdate();
        stmt.close();
        System.out.println("Appointment cancelled successfully!");
    } catch (SQLException e) {
        System.err.println("Error cancelling appointment: " + e.getMessage());
        throw e; }
}

public void completeAppointment(int appointmentId, String visitNotes) throws SQLException {
    String sql = "UPDATE APPOINTMENTS SET STATUS='COMPLETED', VISIT_NOTES=? " +
        "WHERE APPOINTMENT_ID=?";
    try {
        PreparedStatement stmt = connection.prepareStatement(sql);
        stmt.setString(1, visitNotes);
        stmt.setInt(2, appointmentId);
        stmt.executeUpdate();
        stmt.close();
        System.out.println("Appointment marked as completed!");
    } catch (SQLException e) {
        System.err.println("Error completing appointment: " + e.getMessage());
        throw e; }
}

private Appointment extractAppointmentFromResultSet(ResultSet rs) throws SQLException {
    int appointmentId = rs.getInt("APPOINTMENT_ID");
    int patientId = rs.getInt("PATIENT_ID");
    int doctorId = rs.getInt("DOCTOR_ID");
    Date appointmentDate = rs.getDate("APPOINTMENT_DATE");
    LocalDate date = (appointmentDate != null) ? appointmentDate.toLocalDate() : null;
    String time = rs.getString("APPOINTMENT_TIME");
    String status = rs.getString("STATUS");
    String visitNotes = rs.getString("VISIT_NOTES");
    Appointment appointment = new Appointment(appointmentId, patientId, doctorId,
        date, time, status, visitNotes);

    try {
        Patient patient = patientDAO.getByPatientId(patientId);
        Doctor doctor = doctorDAO.getByDoctorId(doctorId);
        appointment.setPatient(patient);
        appointment.setDoctor(doctor);
    } catch (SQLException e) {
        // If loading patient/doctor fails, continue without them
        System.err.println("Warning: Could not load patient/doctor for appointment " + appointmentId);
    }
    return appointment;
}
}
}

```



```

-----
public class PatientDAO implements Manageable<Patient> {
    private Connection connection;
    public PatientDAO() {
        try {
            this.connection = DatabaseConnection.getInstance().getConnection();
        } catch (SQLException e) {
            System.err.println("Failed to get database connection in PatientDAO!");
            e.printStackTrace();
        }
    }
    @Override
    public void add(Patient patient) throws SQLException {
        String userSql = "INSERT INTO USERS (USER_ID, USERNAME, PASSWORD, ROLE, NAME, EMAIL, PHONE) " +
            "VALUES (USER_SEQ.NEXTVAL, ?, ?, 'PATIENT', ?, ?, ?)";
        String patientSql = "INSERT INTO PATIENTS (PATIENT_ID, USER_ID, DATE_OF_BIRTH, BLOOD_GROUP) " +
            "VALUES (PATIENT_SEQ.NEXTVAL, USER_SEQ.CURRVAL, ?, ?)";
        try {
            PreparedStatement userStmt = connection.prepareStatement(userSql);
            userStmt.setString(1, patient.getUsername());
            userStmt.setString(2, patient.getPassword());
            userStmt.setString(3, patient.getName());
            userStmt.setString(4, patient.getEmail());
            userStmt.setString(5, patient.getPhone());
            userStmt.executeUpdate();
            userStmt.close();
            PreparedStatement patientStmt = connection.prepareStatement(patientSql);
            patientStmt.setDate(1, patient.getDateOfBirth() != null ? Date.valueOf(patient.getDateOfBirth()) : null);
            patientStmt.setString(2, patient.getBloodGroup());
            patientStmt.executeUpdate();
            patientStmt.close();
            System.out.println("Patient added successfully!");
        } catch (SQLException e) {
            System.err.println("Error adding patient: " + e.getMessage());
            throw e;
        }
    }
    @Override
    public void update(Patient patient) throws SQLException {
        String userSql = "UPDATE USERS SET USERNAME=?, PASSWORD=?, NAME=?, EMAIL=?, PHONE=? WHERE USER_ID=?";
        String patientSql = "UPDATE PATIENTS SET DATE_OF_BIRTH=?, BLOOD_GROUP=? WHERE USER_ID=?";
        try {
            PreparedStatement userStmt = connection.prepareStatement(userSql);
            userStmt.setString(1, patient.getUsername());
            userStmt.setString(2, patient.getPassword());
            userStmt.setString(3, patient.getName());
            userStmt.setString(4, patient.getEmail());
            userStmt.setString(5, patient.getPhone());
            userStmt.setInt(6, patient.getUserId());
            userStmt.executeUpdate();
            userStmt.close();
            PreparedStatement patientStmt = connection.prepareStatement(patientSql);
            patientStmt.setDate(1, patient.getDateOfBirth() != null ? Date.valueOf(patient.getDateOfBirth()) : null);
            patientStmt.setString(2, patient.getBloodGroup());
            patientStmt.setInt(3, patient.getUserId());
            patientStmt.executeUpdate();
            patientStmt.close();
            System.out.println("Patient updated successfully!");
        } catch (SQLException e) {
            System.err.println("Error updating patient: " + e.getMessage());
            throw e;
        }
    }
    @Override
    public void delete(int userId) throws SQLException {
        String sql = "DELETE FROM USERS WHERE USER_ID=? AND ROLE='PATIENT'";
        try {
            PreparedStatement stmt = connection.prepareStatement(sql);
            stmt.setInt(1, userId);

```

```

        int rowsAffected = stmt.executeUpdate();
        stmt.close();
        if (rowsAffected > 0) System.out.println("Patient deleted successfully!");
        else System.out.println("Patient not found!");
    } catch (SQLException e) {
        System.err.println("Error deleting patient: " + e.getMessage());
        throw e;
    }
}

@Override
public Patient getById(int userId) throws SQLException {
    String sql = "SELECT U.USER_ID, U.USERNAME, U.PASSWORD, U.NAME, U.EMAIL, U.PHONE, " +
        "P.PATIENT_ID, P.DATE_OF_BIRTH, P.BLOOD_GROUP FROM USERS U JOIN PATIENTS P " +
        "ON U.USER_ID = P.USER_ID WHERE U.USER_ID = ?";

    try {
        PreparedStatement stmt = connection.prepareStatement(sql);
        stmt.setInt(1, userId);
        ResultSet rs = stmt.executeQuery();
        Patient patient = null;
        if (rs.next()) patient = extractPatientFromResultSet(rs);
        rs.close();
        stmt.close();
        return patient;
    } catch (SQLException e) {
        System.err.println("Error getting patient by ID: " + e.getMessage());
        throw e;
    }
}

@Override
public ArrayList<Patient> getAll() throws SQLException {
    String sql = "SELECT U.USER_ID, U.USERNAME, U.PASSWORD, U.NAME, U.EMAIL, U.PHONE, " +
        "P.PATIENT_ID, P.DATE_OF_BIRTH, P.BLOOD_GROUP FROM USERS U JOIN PATIENTS P " +
        "ON U.USER_ID = P.USER_ID ORDER BY U.NAME";

    ArrayList<Patient> patients = new ArrayList<>();

    try {
        Statement stmt = connection.createStatement();
        ResultSet rs = stmt.executeQuery(sql);
        while (rs.next()) patients.add(extractPatientFromResultSet(rs));
        rs.close();
        stmt.close();
        return patients;
    } catch (SQLException e) {
        System.err.println("Error getting all patients: " + e.getMessage());
        throw e;
    }
}

public Patient getByUsername(String username) throws SQLException {
    String sql = "SELECT U.USER_ID, U.USERNAME, U.PASSWORD, U.NAME, U.EMAIL, U.PHONE, " +
        "P.PATIENT_ID, P.DATE_OF_BIRTH, P.BLOOD_GROUP FROM USERS U JOIN PATIENTS P " +
        "ON U.USER_ID = P.USER_ID WHERE U.USERNAME = ?";

    try {
        PreparedStatement stmt = connection.prepareStatement(sql);
        stmt.setString(1, username);
        ResultSet rs = stmt.executeQuery();
        Patient patient = null;
        if (rs.next()) patient = extractPatientFromResultSet(rs);
        rs.close();
        stmt.close();
        return patient;
    } catch (SQLException e) {
        System.err.println("Error getting patient by username: " + e.getMessage());
        throw e;
    }
}

public Patient getByPatientId(int patientId) throws SQLException {
    String sql = "SELECT U.USER_ID, U.USERNAME, U.PASSWORD, U.NAME, U.EMAIL, U.PHONE, " +
        "P.PATIENT_ID, P.DATE_OF_BIRTH, P.BLOOD_GROUP FROM USERS U JOIN PATIENTS P " +
        "ON U.USER_ID = P.USER_ID WHERE P.PATIENT_ID = ?";

    try {
        PreparedStatement stmt = connection.prepareStatement(sql);

```

```

        stmt.setInt(1, patientId);
        ResultSet rs = stmt.executeQuery();
        Patient patient = null;
        if (rs.next()) patient = extractPatientFromResultSet(rs);
        rs.close();
        stmt.close();
        return patient;
    } catch (SQLException e) {
        System.err.println("Error getting patient by patient ID: " + e.getMessage());
        throw e;
    }
}

private Patient extractPatientFromResultSet(ResultSet rs) throws SQLException {
    int userId = rs.getInt("USER_ID");
    String username = rs.getString("USERNAME");
    String password = rs.getString("PASSWORD");
    String name = rs.getString("NAME");
    String email = rs.getString("EMAIL");
    String phone = rs.getString("PHONE");
    int patientId = rs.getInt("PATIENT_ID");
    Date dobDate = rs.getDate("DATE_OF_BIRTH");
    LocalDate dob = dobDate != null ? dobDate.toLocalDate() : null;
    String bloodGroup = rs.getString("BLOOD_GROUP");
    return new Patient(userId, username, password, name, email, phone, patientId, dob, bloodGroup);
}

-----

public class DoctorDAO implements Manageable<Doctor>{
    private Connection connection;
    public DoctorDAO(){
        try{
            this.connection=DatabaseConnection.getInstance().getConnection();
        }catch(SQLException e){
            System.err.println("Failed to get database connection in DoctorDAO!");
            e.printStackTrace();
        }
    }
    @Override
    public void add(Doctor doctor)throws SQLException{
        String userSql="INSERT INTO USERS (USER_ID, USERNAME, PASSWORD, ROLE, NAME, EMAIL, PHONE) VALUES
(USER_SEQ.NEXTVAL, ?, ?, 'DOCTOR', ?, ?, ?)";
        String doctorSql="INSERT INTO DOCTORS (DOCTOR_ID, USER_ID, SPECIALTY, EXPERIENCE_YEARS) VALUES
(DOCTOR_SEQ.NEXTVAL, USER_SEQ.CURRVAL, ?, ?)";
        try{
            PreparedStatement userStmt=connection.prepareStatement(userSql);
            userStmt.setString(1,doctor.getUsername());
            userStmt.setString(2,doctor.getPassword());
            userStmt.setString(3,doctor.getName());
            userStmt.setString(4,doctor.getEmail());
            userStmt.setString(5,doctor.getPhone());
            userStmt.executeUpdate();
            userStmt.close();
            PreparedStatement doctorStmt=connection.prepareStatement(doctorSql);
            doctorStmt.setString(1,doctor.getSpecialty());
            doctorStmt.setInt(2,doctor.getExperienceYears());
            doctorStmt.executeUpdate();
            doctorStmt.close();
            System.out.println("Doctor added successfully!");
        }catch(SQLException e){
            System.err.println("Error adding doctor: "+e.getMessage());
            throw e;
        }
    }
    @Override
    public void update(Doctor doctor)throws SQLException{
        String userSql="UPDATE USERS SET USERNAME=?, PASSWORD=?, NAME=?, EMAIL=?, PHONE=? WHERE USER_ID=?";
        String doctorSql="UPDATE DOCTORS SET SPECIALTY=?, EXPERIENCE_YEARS=? WHERE USER_ID=?";
        try{
            PreparedStatement userStmt=connection.prepareStatement(userSql);

```

```

        userStmt.setString(1,doctor.getUsername());
        userStmt.setString(2,doctor.getPassword());
        userStmt.setString(3,doctor.getName());
        userStmt.setString(4,doctor.getEmail());
        userStmt.setString(5,doctor.getPhone());
        userStmt.setInt(6,doctor.getUserId());
        userStmt.executeUpdate();
        userStmt.close();
        PreparedStatement doctorStmt=connection.prepareStatement(doctorSql);
        doctorStmt.setString(1,doctor.getSpecialty());
        doctorStmt.setInt(2,doctor.getExperienceYears());
        doctorStmt.setInt(3,doctor.getUserId());
        doctorStmt.executeUpdate();
        doctorStmt.close();
        System.out.println("Doctor updated successfully!");
    } catch(SQLException e){
        System.err.println("Error updating doctor: "+e.getMessage());
        throw e;
    }
}

@Override
public void delete(int userId)throws SQLException{
    String sql="DELETE FROM USERS WHERE USER_ID=? AND ROLE='DOCTOR'";
    try{
        PreparedStatement stmt=connection.prepareStatement(sql);
        stmt.setInt(1,userId);
        int rowsAffected=stmt.executeUpdate();
        stmt.close();
        if(rowsAffected>0)System.out.println("Doctor deleted successfully!");
        else System.out.println("Doctor not found!");
    } catch(SQLException e){
        System.err.println("Error deleting doctor: "+e.getMessage());
        throw e;
    }
}

@Override
public Doctor getById(int userId)throws SQLException{
    String sql="SELECT U.USER_ID, U.USERNAME, U.PASSWORD, U.NAME, U.EMAIL, U.PHONE, D.DOCTOR_ID, D.SPECIALTY, D.EXPERIENCE_YEARS FROM USERS U JOIN DOCTORS D ON U.USER_ID = D.USER_ID WHERE U.USER_ID = ?";
    try{
        PreparedStatement stmt=connection.prepareStatement(sql);
        stmt.setInt(1,userId);
        ResultSet rs=stmt.executeQuery();
        Doctor doctor=null;
        if(rs.next())doctor=extractDoctorFromResultSet(rs);
        rs.close();
        stmt.close();
        return doctor;
    } catch(SQLException e){
        System.err.println("Error getting doctor by ID: "+e.getMessage());
        throw e;
    }
}

@Override
public ArrayList<Doctor> getAll()throws SQLException{
    String sql="SELECT U.USER_ID, U.USERNAME, U.PASSWORD, U.NAME, U.EMAIL, U.PHONE, D.DOCTOR_ID, D.SPECIALTY, D.EXPERIENCE_YEARS FROM USERS U JOIN DOCTORS D ON U.USER_ID = D.USER_ID ORDER BY U.NAME";
    ArrayList<Doctor> doctors=new ArrayList<>();
    try{
        Statement stmt=connection.createStatement();
        ResultSet rs=stmt.executeQuery(sql);
        while(rs.next())doctors.add(extractDoctorFromResultSet(rs));
        rs.close();
        stmt.close();
        return doctors;
    } catch(SQLException e){
        System.err.println("Error getting all doctors: "+e.getMessage());
        throw e;
    }
}
}

```

```

public Doctor getByUsername(String username)throws SQLException{
    String sql="SELECT U.USER_ID, U.USERNAME, U.PASSWORD, U.NAME, U.EMAIL, U.PHONE, D.DOCTOR_ID, D.SPECIALTY,
D.EXPERIENCE_YEARS FROM USERS U JOIN DOCTORS D ON U.USER_ID = D.USER_ID WHERE U.USERNAME = ?";
    try{
        PreparedStatement stmt=connection.prepareStatement(sql);
        stmt.setString(1,username);
        ResultSet rs=stmt.executeQuery();
        Doctor doctor=null;
        if(rs.next())doctor=extractDoctorFromResultSet(rs);
        rs.close();
        stmt.close();
        return doctor;
    }catch(SQLException e){
        System.err.println("Error getting doctor by username: "+e.getMessage());
        throw e;
    }
}

public Doctor getByDoctorId(int doctorId)throws SQLException{
    String sql="SELECT U.USER_ID, U.USERNAME, U.PASSWORD, U.NAME, U.EMAIL, U.PHONE, D.DOCTOR_ID, D.SPECIALTY,
D.EXPERIENCE_YEARS FROM USERS U JOIN DOCTORS D ON U.USER_ID = D.USER_ID WHERE D.DOCTOR_ID = ?";
    try{
        PreparedStatement stmt=connection.prepareStatement(sql);
        stmt.setInt(1,doctorId);
        ResultSet rs=stmt.executeQuery();
        Doctor doctor=null;
        if(rs.next())doctor=extractDoctorFromResultSet(rs);
        rs.close();
        stmt.close();
        return doctor;
    }catch(SQLException e){
        System.err.println("Error getting doctor by doctor ID: "+e.getMessage());
        throw e;
    }
}

public ArrayList<Doctor> getBySpecialty(String specialty)throws SQLException{
    String sql="SELECT U.USER_ID, U.USERNAME, U.PASSWORD, U.NAME, U.EMAIL, U.PHONE, D.DOCTOR_ID, D.SPECIALTY,
D.EXPERIENCE_YEARS FROM USERS U JOIN DOCTORS D ON U.USER_ID = D.USER_ID WHERE D.SPECIALTY = ? ORDER BY
U.NAME";
    ArrayList<Doctor> doctors=new ArrayList<>();
    try{
        PreparedStatement stmt=connection.prepareStatement(sql);
        stmt.setString(1,specialty);
        ResultSet rs=stmt.executeQuery();
        while(rs.next())doctors.add(extractDoctorFromResultSet(rs));
        rs.close();
        stmt.close();
        return doctors;
    }catch(SQLException e){
        System.err.println("Error getting doctors by specialty: "+e.getMessage());
        throw e;
    }
}

public ArrayList<String> getAllSpecialties()throws SQLException{
    String sql="SELECT DISTINCT SPECIALTY FROM DOCTORS ORDER BY SPECIALTY";
    ArrayList<String> specialties=new ArrayList<>();
    try{
        Statement stmt=connection.createStatement();
        ResultSet rs=stmt.executeQuery(sql);
        while(rs.next())specialties.add(rs.getString("SPECIALTY"));
        rs.close();
        stmt.close();
        return specialties;
    }catch(SQLException e){
        System.err.println("Error getting specialties: "+e.getMessage());
        throw e;
    }
}

private Doctor extractDoctorFromResultSet(ResultSet rs)throws SQLException{
    int userId=rs.getInt("USER_ID");
    String username=rs.getString("USERNAME");

```

```

        String password=rs.getString("PASSWORD");
        String name=rs.getString("NAME");
        String email=rs.getString("EMAIL");
        String phone=rs.getString("PHONE");
        int doctorId=rs.getInt("DOCTOR_ID");
        String specialty=rs.getString("SPECIALTY");
        int experienceYears=rs.getInt("EXPERIENCE_YEARS");
        return new Doctor(userId,username,password,name,email,phone,doctorId,specialty,experienceYears);
    }
}

```

```

-----
public class DatabaseConnection{
    private static DatabaseConnection instance;
    private Connection connection;
    private static final String DB_URL="jdbc:oracle:thin:@localhost:1521:XE";
    private static final String DB_USER="SYSTEM";
    private static final String DB_PASSWORD="SYSTEM2005";
    private DatabaseConnection(){
        try{
            Class.forName("oracle.jdbc.driver.OracleDriver");
            connection=DriverManager.getConnection(DB_URL,DB_USER,DB_PASSWORD);
            System.out.println("Database connection established successfully!");
        }catch(ClassNotFoundException e){
            System.err.println("Oracle JDBC Driver not found!");
            System.err.println("Please add ojdbc8.jar to the project library.");
            e.printStackTrace();
        }catch(SQLException e){
            System.err.println("Failed to connect to database!");
            System.err.println("Error: "+e.getMessage());
            e.printStackTrace();
        }
    }
    public static synchronized DatabaseConnection getInstance(){
        if(instance==null){
            instance=new DatabaseConnection();
        }
        return instance;
    }
    public Connection getConnection()throws SQLException{
        if(connection==null||connection.isClosed()){
            try{
                connection=DriverManager.getConnection(DB_URL,DB_USER,DB_PASSWORD);
            }catch(SQLException e){
                System.err.println("Failed to re-establish database connection!");
                throw e;
            }
        }
        return connection;
    }
    public void closeConnection(){
        try{
            if(connection!=null&&!connection.isClosed()){
                connection.close();
                System.out.println("Database connection closed.");
            }
        }catch(SQLException e){
            System.err.println("Error closing database connection!");
            e.printStackTrace();
        }
    }
    public boolean testConnection(){
        try{
            Connection conn=getConnection();
            return conn!=null&&!conn.isClosed();
        }catch(SQLException e){
            System.err.println("Connection test failed!");
            e.printStackTrace();
            return false;
        }
    }
}

```

```

}
-----
public class LoginController{
    @FXML private TextField usernameField;
    @FXML private PasswordField passwordField;
    @FXML private Label statusLabel;
    private PatientDAO patientDAO;
    private DoctorDAO doctorDAO;
    @FXML
    public void initialize(){
        patientDAO=new PatientDAO();
        doctorDAO=new DoctorDAO();
        statusLabel.setText("");
    }
    @FXML
    private void handleLogin(ActionEvent event){
        String username=usernameField.getText().trim();
        String password=passwordField.getText();
        if(username.isEmpty()||password.isEmpty()){
            statusLabel.setText("Please enter username and password");
            AlertHelper.showEmptyFieldsError();
            return;
        }
        try{
            User user=authenticateUser(username,password);
            if(user!=null){
                SessionManager.getInstance().setCurrentUser(user);
                statusLabel.setText("Login successful!");
                redirectToDashboard(event,user);
            }else{
                statusLabel.setText("Invalid username or password");
                AlertHelper.showLoginError();
            }
        }catch(SQLException e){
            statusLabel.setText("Database error occurred");
            AlertHelper.showDatabaseError(e.getMessage());
            e.printStackTrace();
        }catch(Exception e){
            statusLabel.setText("An error occurred");
            AlertHelper.showError("Error","Login Error",e.getMessage());
            e.printStackTrace();
        }
    }
    @FXML
    private void handleRegister(ActionEvent event){
        try{
            FXMLLoader loader=new FXMLLoader(getClass().getResource("/projectcode1/views/RegisterView.fxml"));
            Parent root=loader.load();
            Stage stage=(Stage)((Node)event.getSource()).getScene().getWindow();
            Scene scene=new Scene(root);
            stage.setScene(scene);
            stage.setTitle("Patient Registration");
            stage.show();
        }catch(Exception e){
            AlertHelper.showError("Error","Navigation Error","Could not load registration screen");
            e.printStackTrace();
        }
    }
    private User authenticateUser(String username,String password)throws SQLException{
        String sql="SELECT USER_ID, ROLE FROM USERS WHERE USERNAME = ? AND PASSWORD = ?";
        Connection connection=DatabaseConnection.getInstance().getConnection();
        PreparedStatement stmt=connection.prepareStatement(sql);
        stmt.setString(1,username);
        stmt.setString(2,password);
        ResultSet rs=stmt.executeQuery();
        User user=null;
        if(rs.next()){
            int userId=rs.getInt("USER_ID");
            String role=rs.getString("ROLE");
            if("PATIENT".equals(role)){
                user=patientDAO.getById(userId);
            }
        }
    }
}

```

```

        }else if("DOCTOR".equals(role)){
            user=doctorDAO.getById(userId);
        }else if("ADMIN".equals(role)){
            user=loadAdminUser(userId);
        }
    }
    rs.close();
    stmt.close();
    return user;
}
private User loadAdminUser(int userId)throws SQLException{
    String sql="SELECT * FROM USERS WHERE USER_ID = ?";
    Connection connection=DatabaseConnection.getInstance().getConnection();
    PreparedStatement stmt=connection.prepareStatement(sql);
    stmt.setInt(1,userId);
    ResultSet rs=stmt.executeQuery();
    projectcode1.models.Admin admin=null;
    if(rs.next()){
        String username=rs.getString("USERNAME");
        String password=rs.getString("PASSWORD");
        String name=rs.getString("NAME");
        String email=rs.getString("EMAIL");
        String phone=rs.getString("PHONE");
        admin=new projectcode1.models.Admin(userId,username,password,name,email,phone,1);
    }
    rs.close();
    stmt.close();
    return admin;
}
private void redirectToDashboard(ActionEvent event,User user){
    try{
        String fxmlFile="";
        if(user instanceof Patient){
            fxmlFile="/projectcode1/views/PatientDashboard.fxml";
        }else if(user instanceof Doctor){
            fxmlFile="/projectcode1/views/DoctorDashboard.fxml";
        }else{
            fxmlFile="/projectcode1/views/AdminDashboard.fxml";
        }
        FXMLLoader loader=new FXMLLoader(getClass().getResource(fxmlFile));
        Parent root=loader.load();
        Stage stage=(Stage)((Node)event.getSource()).getScene().getWindow();
        Scene scene=new Scene(root);
        stage.setScene(scene);
        stage.setTitle("Dashboard - "+user.getName());
        stage.show();
    }catch(Exception e){
        AlertHelper.showError("Error","Navigation Error","Could not load dashboard: "+e.getMessage());
        e.printStackTrace();
    }
}
}-----

public class RegisterController{
    @FXML private TextField usernameField;
    @FXML private PasswordField passwordField;
    @FXML private TextField nameField;
    @FXML private TextField emailField;
    @FXML private TextField phoneField;
    @FXML private DatePicker dobPicker;
    @FXML private ComboBox<String> bloodGroupCombo;
    @FXML private Label statusLabel;
    @FXML
    public void initialize(){
        statusLabel.setText("");
        bloodGroupCombo.getItems().addAll("A+","A-","B+","B-","AB+","AB-","O+","O-");
    }
    @FXML
    private void handleRegister(ActionEvent event){
        String username=usernameField.getText().trim();
        String password=passwordField.getText();

```



```

String name=nameField.getText().trim();
String email=emailField.getText().trim();
String phone=phoneField.getText().trim();
LocalDate dob=dobPicker.getValue();
String bloodGroup=bloodGroupCombo.getValue();
if(username.isEmpty()||password.isEmpty()||name.isEmpty()||email.isEmpty()||phone.isEmpty()){
    statusLabel.setText("Please fill in all required fields");
    AlertHelper.showEmptyFieldsError();
    return;
}
if(dob==null){
    statusLabel.setText("Please select date of birth");
    AlertHelper.showValidationError("Please select your date of birth");
    return;
}
if(bloodGroup==null){
    statusLabel.setText("Please select blood group");
    AlertHelper.showValidationError("Please select your blood group");
    return;
}
if(password.length()<6){
    statusLabel.setText("Password must be at least 6 characters");
    AlertHelper.showValidationError("Password must be at least 6 characters long");
    return;
}
try{
    if(usernameExists(username)){
        statusLabel.setText("Username already exists");
        AlertHelper.showWarning("Registration Failed", "Username Taken", "This username is already in use. Please choose another.");
        return;
    }
    registerPatient(username,password,name,email,phone,dob,bloodGroup);
    statusLabel.setText("Registration successful!");
    AlertHelper.showRegistrationSuccess();
    handleBack(event);
}catch(SQLException e){
    statusLabel.setText("Database error occurred");
    AlertHelper.showDatabaseError("Registration failed: "+e.getMessage());
    e.printStackTrace();
}catch(Exception e){
    statusLabel.setText("An error occurred");
    AlertHelper.showError("Error", "Registration Error", e.getMessage());
    e.printStackTrace();
}
}
@FXML
private void handleBack(ActionEvent event){
    try{
        FXMLLoader loader=new FXMLLoader(getClass().getResource("/projectcode1/views/LoginView.fxml"));
        Parent root=loader.load();
        Stage stage=(Stage)((Node)event.getSource()).getScene().getWindow();
        Scene scene=new Scene(root);
        stage.setScene(scene);
        stage.setTitle("Hospital Appointment System - Login");
        stage.show();
    }catch(Exception e){
        AlertHelper.showError("Error", "Navigation Error", "Could not load login screen");
        e.printStackTrace();
    }
}
private boolean usernameExists(String username)throws SQLException{
    String sql="SELECT COUNT(*) FROM USERS WHERE USERNAME = ?";
    Connection connection=DatabaseConnection.getInstance().getConnection();
    PreparedStatement stmt=connection.prepareStatement(sql);
    stmt.setString(1,username);
    ResultSet rs=stmt.executeQuery();
    boolean exists=false;
    if(rs.next()){ exists=rs.getInt(1)>0; }
    rs.close();
    stmt.close();
    return exists;
}

```

```

    }
    private void registerPatient(String username,String password,String name,String email,String phone,LocalDate dob,String bloodGroup)throws
    SQLException{
        String userSql="INSERT INTO USERS (USER_ID, USERNAME, PASSWORD, ROLE, NAME, EMAIL, PHONE) VALUES
        (USER_SEQ.NEXTVAL, ?, ?, 'PATIENT', ?, ?, ?)";
        String patientSql="INSERT INTO PATIENTS (PATIENT_ID, USER_ID, DATE_OF_BIRTH, BLOOD_GROUP) VALUES
        (PATIENT_SEQ.NEXTVAL, USER_SEQ.CURRVAL, ?, ?)";
        Connection connection=DatabaseConnection.getInstance().getConnection();
        try{
            PreparedStatement userStmt=connection.prepareStatement(userSql);
            userStmt.setString(1,username);
            userStmt.setString(2,password);
            userStmt.setString(3,name);
            userStmt.setString(4,email);
            userStmt.setString(5,phone);
            userStmt.executeUpdate();
            userStmt.close();
            PreparedStatement patientStmt=connection.prepareStatement(patientSql);
            patientStmt.setDate(1,Date.valueOf(dob));
            patientStmt.setString(2,bloodGroup);
            patientStmt.executeUpdate();
            patientStmt.close();
            System.out.println("Patient registered successfully: "+username);
        }catch(SQLException e){
            System.err.println("Error registering patient: "+e.getMessage());
            throw e;
        }
    }
}

```

```

-----

public class AdminDashboardController{
    @FXML private Label welcomeLabel;
    @FXML
    public void initialize(){
        User admin=SessionManager.getInstance().getCurrentUser();
        if(admin!=null){
            welcomeLabel.setText("Welcome, "+admin.getName()+"!");
        }
    }
    @FXML
    private void handleManageUsers(ActionEvent event){
        try{
            FXMLLoader loader=new FXMLLoader(getClass().getResource("/projectcode1/views/ManageUsers.fxml"));
            Parent root=loader.load();
            Stage stage=(Stage)((Node)event.getSource()).getScene().getWindow();
            Scene scene=new Scene(root);
            stage.setScene(scene);
            stage.setTitle("Manage Users");
            stage.show();
        }catch(Exception e){
            AlertHelper.showError("Error","Navigation Error","Could not load Manage Users screen");
            e.printStackTrace();
        }
    }
    @FXML
    private void handleViewAllAppointments(ActionEvent event){
        try{
            FXMLLoader loader=new FXMLLoader(getClass().getResource("/projectcode1/views/ViewAllAppointments.fxml"));
            Parent root=loader.load();
            Stage stage=(Stage)((Node)event.getSource()).getScene().getWindow();
            Scene scene=new Scene(root);
            stage.setScene(scene);
            stage.setTitle("All Appointments");
            stage.show();
        }catch(Exception e){
            AlertHelper.showError("Error","Navigation Error","Could not load Appointments screen");
            e.printStackTrace();
        }
    }
}

```

```

@FXML
private void handleLogout(ActionEvent event){
    try{
        SessionManager.getInstance().logout();
        FXMLLoader loader=new FXMLLoader(getClass().getResource("/projectcode1/views/LoginView.fxml"));
        Parent root=loader.load();
        Stage stage=(Stage)((Node)event.getSource()).getScene().getWindow();
        Scene scene=new Scene(root);
        stage.setScene(scene);
        stage.setTitle("Hospital Appointment System - Login");
        stage.show();
    }catch(Exception e){
        AlertHelper.showError("Error","Navigation Error","Could not logout");
        e.printStackTrace();
    }
}
}
}
-----

public class PatientDashboardController{
    @FXML private Label welcomeLabel;
    @FXML
    public void initialize(){
        Patient patient=SessionManager.getInstance().getCurrentPatient();
        if(patient!=null){
            welcomeLabel.setText("Welcome, "+patient.getName()+"!");
        }
    }
    @FXML
    private void handleBrowseDoctors(ActionEvent event){
        try{
            FXMLLoader loader=new FXMLLoader(getClass().getResource("/projectcode1/views/BrowseDoctors.fxml"));
            Parent root=loader.load();
            Stage stage=(Stage)((Node)event.getSource()).getScene().getWindow();
            Scene scene=new Scene(root);
            stage.setScene(scene);
            stage.setTitle("Browse Doctors");
            stage.show();
        }catch(Exception e){
            AlertHelper.showError("Error","Navigation Error","Could not load Browse Doctors screen");
            e.printStackTrace();
        }
    }
    @FXML
    private void handleMyAppointments(ActionEvent event){
        try{
            FXMLLoader loader=new FXMLLoader(getClass().getResource("/projectcode1/views/PatientAppointments.fxml"));
            Parent root=loader.load();
            Stage stage=(Stage)((Node)event.getSource()).getScene().getWindow();
            Scene scene=new Scene(root);
            stage.setScene(scene);
            stage.setTitle("My Appointments");
            stage.show();
        }catch(Exception e){
            AlertHelper.showError("Error","Navigation Error","Could not load Appointments screen");
            e.printStackTrace();
        }
    }
    @FXML
    private void handleLogout(ActionEvent event){
        try{
            SessionManager.getInstance().logout();
            FXMLLoader loader=new FXMLLoader(getClass().getResource("/projectcode1/views/LoginView.fxml"));
            Parent root=loader.load();
            Stage stage=(Stage)((Node)event.getSource()).getScene().getWindow();
            Scene scene=new Scene(root);
            stage.setScene(scene);
            stage.setTitle("Hospital Appointment System - Login");
            stage.show();
        }catch(Exception e){
            AlertHelper.showError("Error","Navigation Error","Could not logout");

```

```

        e.printStackTrace();
    }
}
}

-----

public class DoctorDashboardController{
    @FXML private Label welcomeLabel;
    @FXML private Label specialtyLabel;
    @FXML private Label experienceLabel;
    @FXML
    public void initialize(){
        Doctor doctor=SessionManager.getInstance().getCurrentDoctor();
        if(doctor!=null){
            welcomeLabel.setText("Welcome, Dr. "+doctor.getName()+"!");
            specialtyLabel.setText("Specialty: "+doctor.getSpecialty());
            experienceLabel.setText("Experience: "+doctor.getExperienceYears()+" years");
        }
    }
    @FXML
    private void handleViewSchedule(ActionEvent event){
        try{
            FXMLLoader loader=new FXMLLoader(getClass().getResource("/projectcode1/views/DoctorSchedule.fxml"));
            Parent root=loader.load();
            Stage stage=(Stage)((Node)event.getSource()).getScene().getWindow();
            Scene scene=new Scene(root);
            stage.setScene(scene);
            stage.setTitle("My Schedule");
            stage.show();
        }catch(Exception e){
            AlertHelper.showError("Error","Navigation Error","Could not load schedule screen");
            e.printStackTrace();
        }
    }
    @FXML
    private void handleLogout(ActionEvent event){
        try{
            SessionManager.getInstance().logout();
            FXMLLoader loader=new FXMLLoader(getClass().getResource("/projectcode1/views/LoginView.fxml"));
            Parent root=loader.load();
            Stage stage=(Stage)((Node)event.getSource()).getScene().getWindow();
            Scene scene=new Scene(root);
            stage.setScene(scene);
            stage.setTitle("Hospital Appointment System - Login");
            stage.show();
        }catch(Exception e){
            AlertHelper.showError("Error","Navigation Error","Could not logout");
            e.printStackTrace();
        }
    }
}
}

-----

public class AlertHelper{
    public static void showInfo(String title,String header,String content){
        Alert alert=new Alert(AlertType.INFORMATION);
        alert.setTitle(title);
        alert.setHeaderText(header);
        alert.setContentText(content);
        alert.showAndWait();
    }
    public static void showWarning(String title,String header,String content){
        Alert alert=new Alert(AlertType.WARNING);
        alert.setTitle(title);
        alert.setHeaderText(header);
        alert.setContentText(content);
        alert.showAndWait();
    }
    public static void showError(String title,String header,String content){

```

```

        Alert alert=new Alert(AlertType.ERROR);
        alert.setTitle(title);
        alert.setHeaderText(header);
        alert.setContentText(content);
        alert.showAndWait();
    }
    public static void showSuccess(String title,String header,String message){
        showInfo(title,header,message);
    }
    public static void showDatabaseError(String message){
        showError("Database Error","A database error occurred",message);
    }
    public static void showValidationError(String message){
        showWarning("Validation Error","Please check your input",message);
    }
    public static boolean showConfirmation(String title,String header,String content){
        Alert alert=new Alert(AlertType.CONFIRMATION);
        alert.setTitle(title);
        alert.setHeaderText(header);
        alert.setContentText(content);
        Optional<ButtonType> result=alert.showAndWait();
        return result.isPresent()&&result.get()==ButtonType.OK;
    }
    public static boolean showDeleteConfirmation(String itemName){
        return showConfirmation("Confirm Deletion","Are you sure you want to delete this item?","Deleting: "+itemName+"\n\nThis action cannot
be undone.");
    }
    public static boolean showCancelConfirmation(String itemName){
        return showConfirmation("Confirm Cancellation","Are you sure you want to cancel?","Cancelling: "+itemName);
    }
    public static void showLoginError(){
        showError("Login Failed","Invalid Credentials","The username or password you entered is incorrect.\nPlease try again.");
    }
    public static void showRegistrationSuccess(){
        showSuccess("Registration Successful","Success!","Your account has been created successfully!\nYou can now log in.");
    }
    public static void showEmptyFieldsError(){
        showValidationError("Please fill in all required fields.");
    }
    public static void showAppointmentBooked(){
        showSuccess("Appointment Booked","Success!","Your appointment has been scheduled successfully!");
    }
    public static void showAppointmentCancelled(){
        showSuccess("Appointment Cancelled","Success!","The appointment has been cancelled.");
    }
    public static void showAppointmentCompleted(){
        showSuccess("Appointment Completed","Success!","The appointment has been marked as completed with visit notes.");
    }
}

```

```

-----

public class SessionManager{
    private static SessionManager instance;
    private User currentUser;
    private SessionManager(){}
    public static SessionManager getInstance(){
        if(instance==null) instance=new SessionManager();
        return instance;
    }
    public void setCurrentUser(User user){
        this.currentUser=user;
        System.out.println("User logged in: "+user.getName()+" (" +user.getRole()+)");
    }
    public User getCurrentUser(){return currentUser;}
    public Patient getCurrentPatient(){return currentUser instanceof Patient?(Patient)currentUser:null;}
    public Doctor getCurrentDoctor(){return currentUser instanceof Doctor?(Doctor)currentUser:null;}
    public Admin getCurrentAdmin(){return currentUser instanceof Admin?(Admin)currentUser:null;}
    public boolean isLoggedIn(){return currentUser!=null;}
    public boolean isPatient(){return currentUser instanceof Patient;}
    public boolean isDoctor(){return currentUser instanceof Doctor;}
}

```

```
public boolean isAdmin() {return currentUser instanceof Admin;}
public String getCurrentRole() {return currentUser!=null?currentUser.getRole():null;}
public void logout() {
    if(currentUser!=null) {
        System.out.println("User logged out: "+currentUser.getName());
        currentUser=null;
    }
}
public void clearSession() {logout();}
}
```